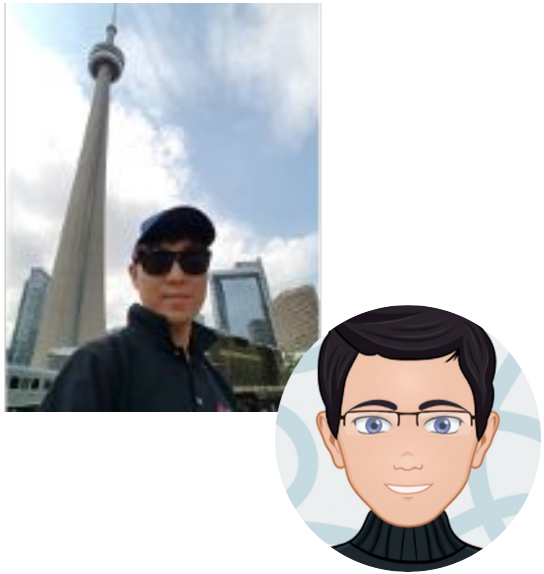
KServe 101

An Introduction to CNCF's New Model Serving Project



Jooho Lee
Principal Software Engineer at Red Hat



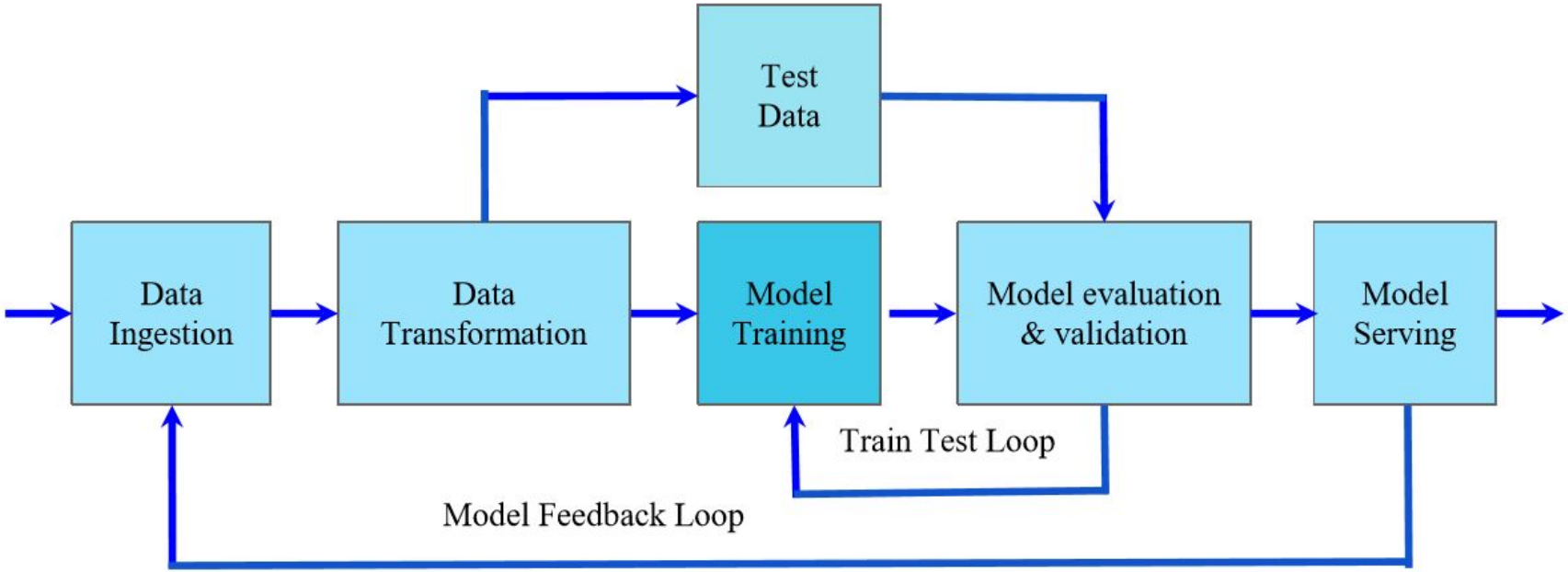
Jooho Lee (Jay)

Jooho Lee is a Toronto-based technology professional with over a decade at Red Hat, focusing on cloud-native platforms and AI. He began in middleware support and later consulted on OpenShift, helping customers design and operate Kubernetes platforms. Now on the OpenShift AI Model Serving team, he integrates KServe and LLM-D into Open Data Hub for production-grade, Kubernetes-native AI/ML deployments. He is also a KServe committer, contributing to model serving and LLM inference..

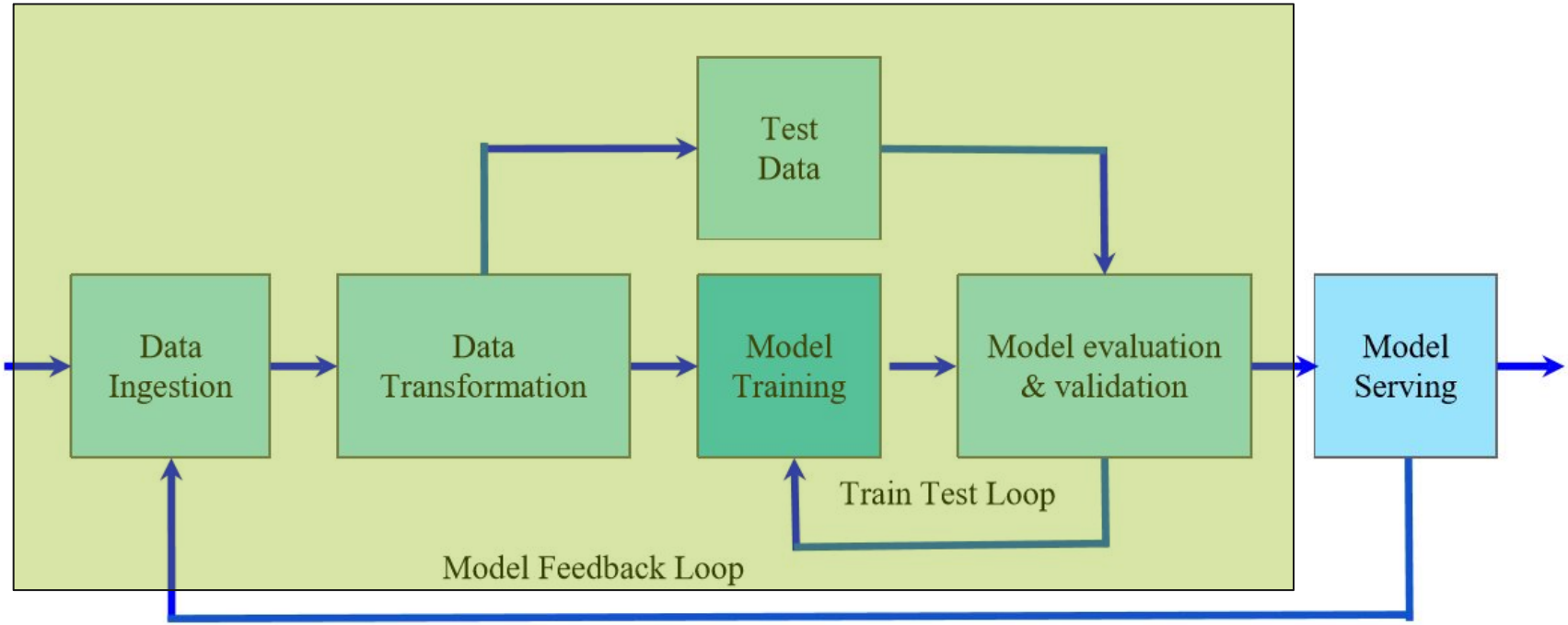
LinkedIn: www.linkedin.com/in/jooho
Github:https://github.com/jooho

High Level Understanding

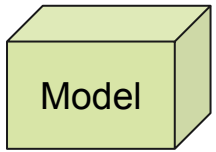
Model Lifecycle



Training

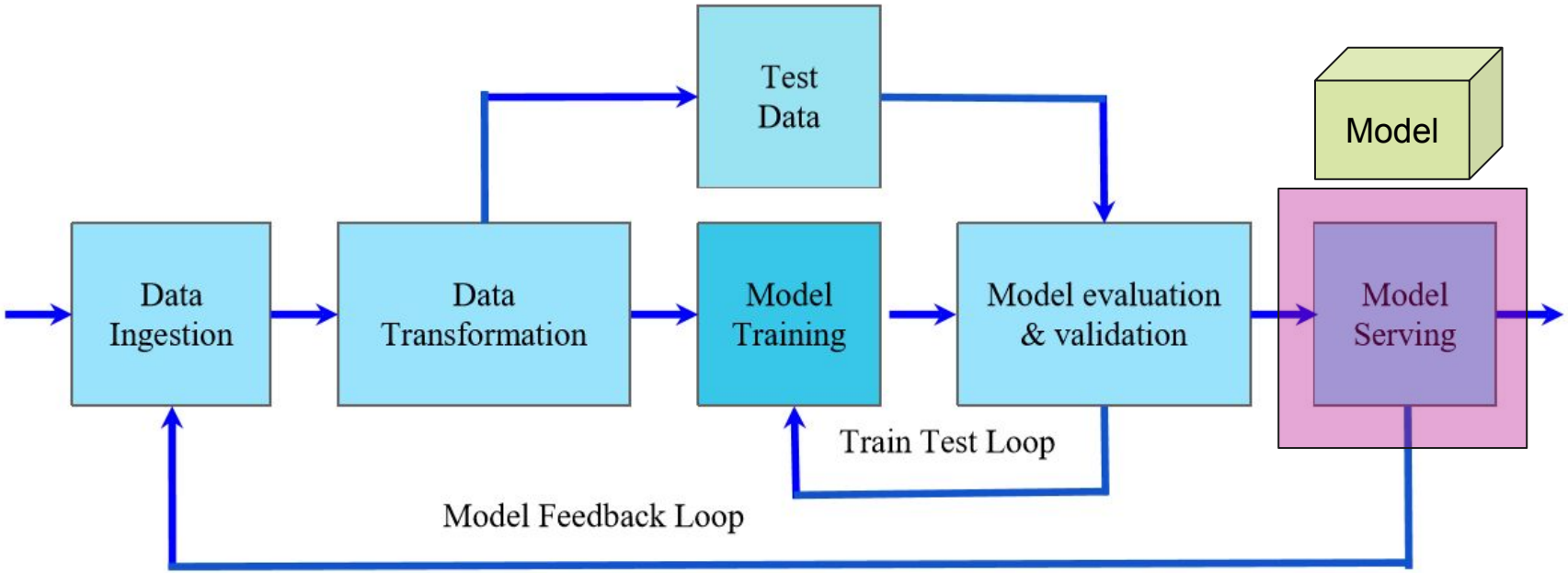


It is for GENERATING a Model

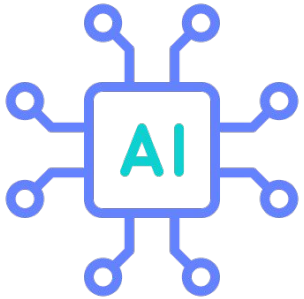


Inferencing

It is for USING Models







How do you deploy a model for Service on Kubernetes?

Custom image?

Kubernetes-native ML platform?

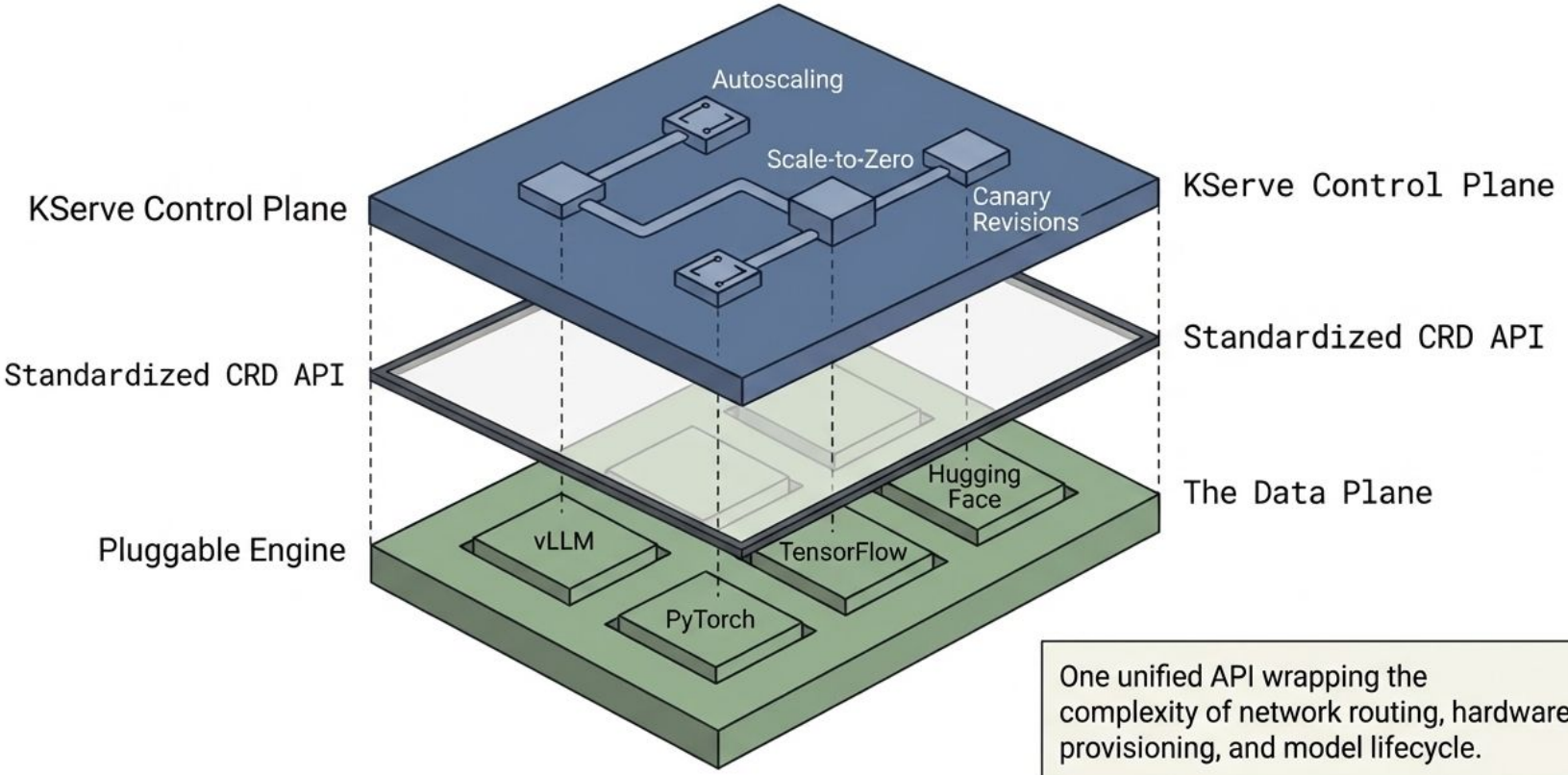
ModelServing Framework?

KServe

What is KServe?

Standardized Distributed Generative and Predictive AI Inference Platform for Scalable, Multi-Framework Deployment on Kubernetes

The Cloud-Native Unifier



One unified API wrapping the complexity of network routing, hardware provisioning, and model lifecycle.

KServe Deployment Types

- RawDeployment
  - KServe + Cert Manager
- Serverless
  - KServe + Knative + ISTIO + Cert Manager
  - KService + Knative + Alternative networking layer (Kourier) + Cert Manager
- ModelMesh
  - ModelMesh Serving



RawDeployment

- KServe + Cert Manager



Serverless

- KServe + Knative + ISTIO + Cert Manager



ModelMesh

- ModelMesh Serving

Why is KServe?

Predictive AI: Forecasting Future Outcomes



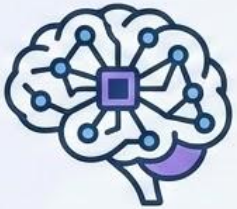
Predictive Model

From Historical Data to Future Insights:
Trends, Patterns, and Decisions.

Large Language Models (LLMs): Powering Generative AI



Massive Text Data



Large Language Model (LLM)



Text Generation
(Creative writing, code, emails)



Translation
(Multi-language support)

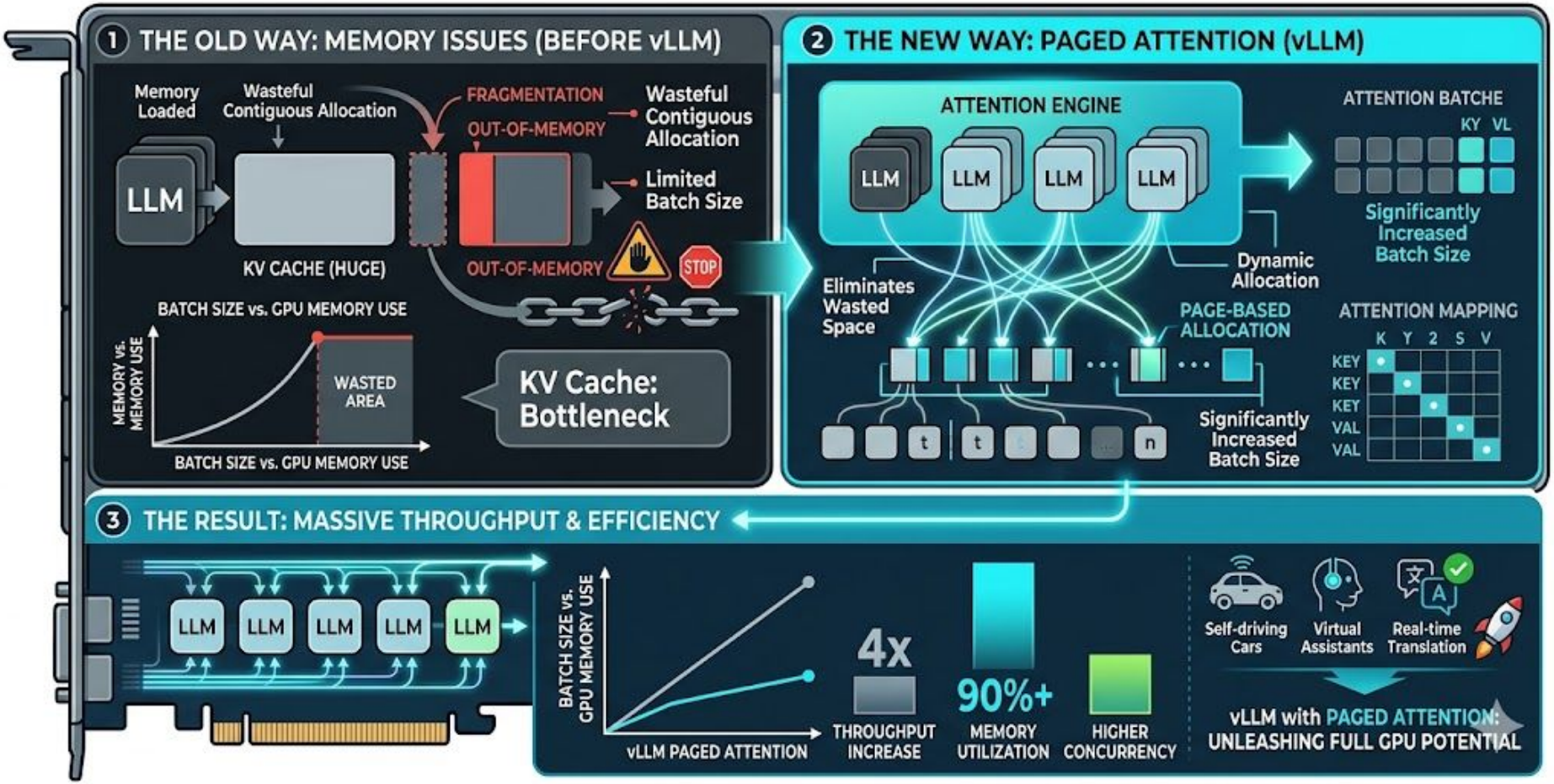


Summarization
(Condensing information)



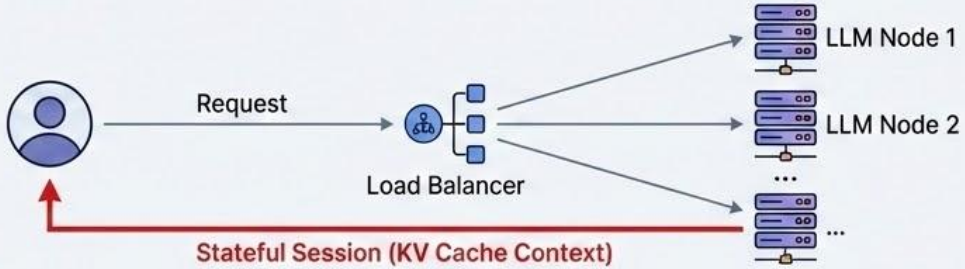
Question Answering
(Chatbots, virtual assistants)

HOW vLLM's PAGED ATTENTION REVOLUTIONIZES GPU MEMORY



The Statefulness Challenge: Scaling LLMs is Hard

Unlike stateless web requests, LLM queries rely on a heavy KV Cache that must be maintained across multiple tokens.





1. The 'KV Cache' Bottleneck

vLLM's PagedAttention manages memory within one node, but scaling across nodes causes fragmentation.

High concurrency leads to "preemption" (evicting active sessions), causing massive spikes in Time Per Output Token (TPOT).



2. Inefficient Load Balancing

Generic L4/L7 load balancers are "LLM-blind". Standard Round-Robin ignores actual GPU memory pressure or queue length of individual vLLM instances.

Result: Some nodes idle, others overwhelmed, collapsing throughput.



3. Lack of Request Prioritization

vLLM processes requests mostly first-come, first-served.

Production needs prioritizing short interactive chats over long batch processing to maintain consistent User Experience (UX).



4. Scalability & Observability Gaps

vLLM focuses on Inference Execution; lacks built-in global rate limiting, cost tracking, or seamless failover across multiple GPU clusters.

The Solution: LLM-native Load Balancing (llmd)

Session Awareness & KV Cache Affinity

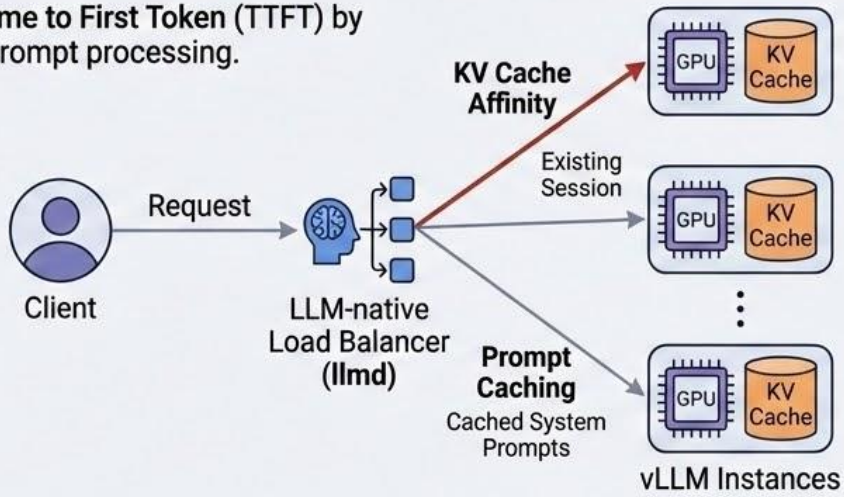
Routes requests to the same vLLM instance where the previous KV Cache resides.

Significantly **reduces Time to First Token** (TTFT) by minimizing redundant prompt processing.

Prompt Caching Optimization

Identifies common system prompts or prefixes and routes them to nodes where those prompts are already cached.

Maximizes System Throughput by reusing memory across different user sessions.



Dynamic Load Shedding & Queue Management

Monitors real-time GPU utilization and queue depth of each vLLM node.

Prevents Request Overload by intelligently rerouting or holding traffic before it hits the inference engine.

Enhanced Observability & Reliability

Provides granular metrics for LLM performance (TPOT, TTFT, Token Usage).

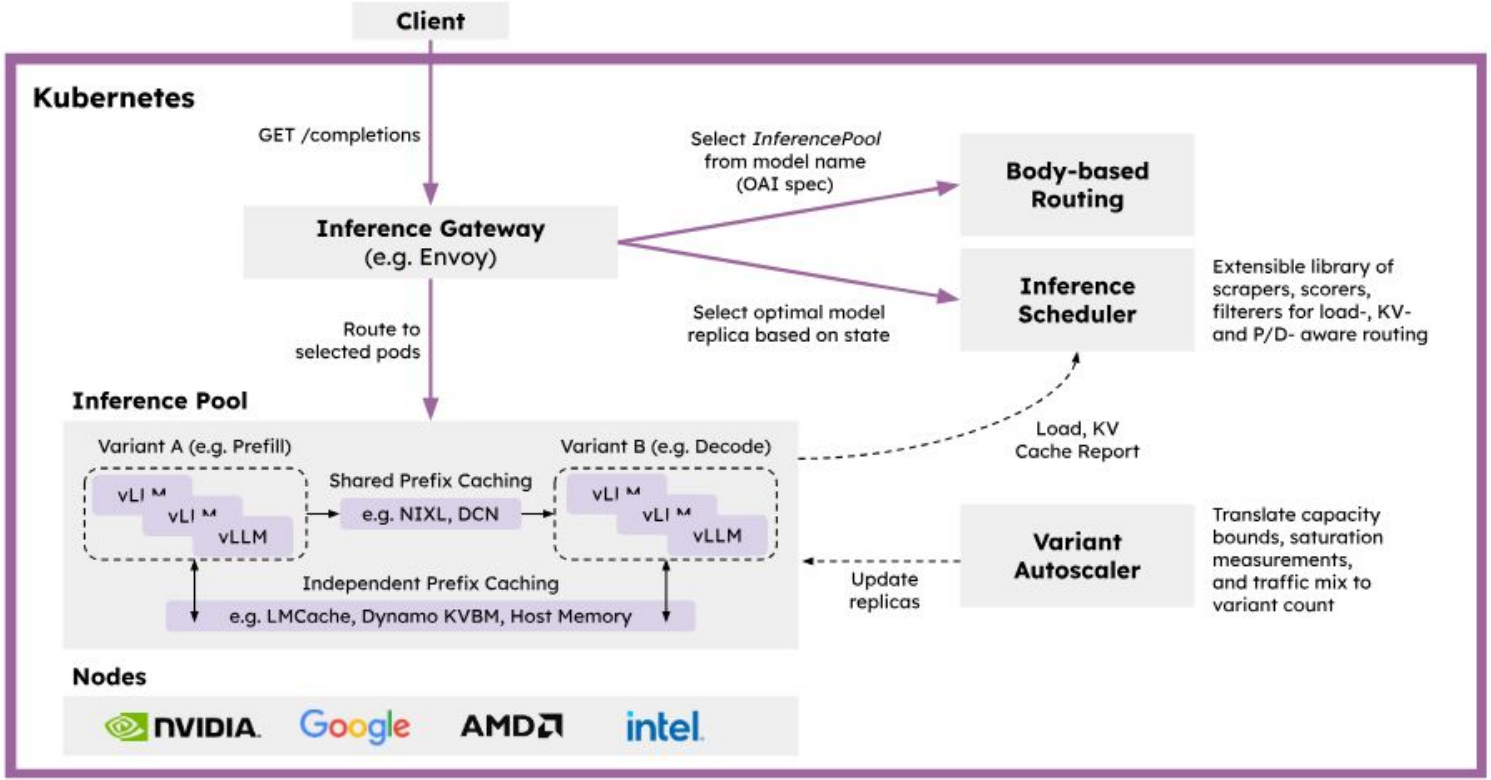
Ensures High Availability through seamless failover and health checks specifically designed for GPU-backed services.

Priority-based Scheduling

Implements different service tiers to ensure low latency for premium or interactive users.

Balances Throughput vs. Latency based on business requirements and workload types.

LLM-D Architecture



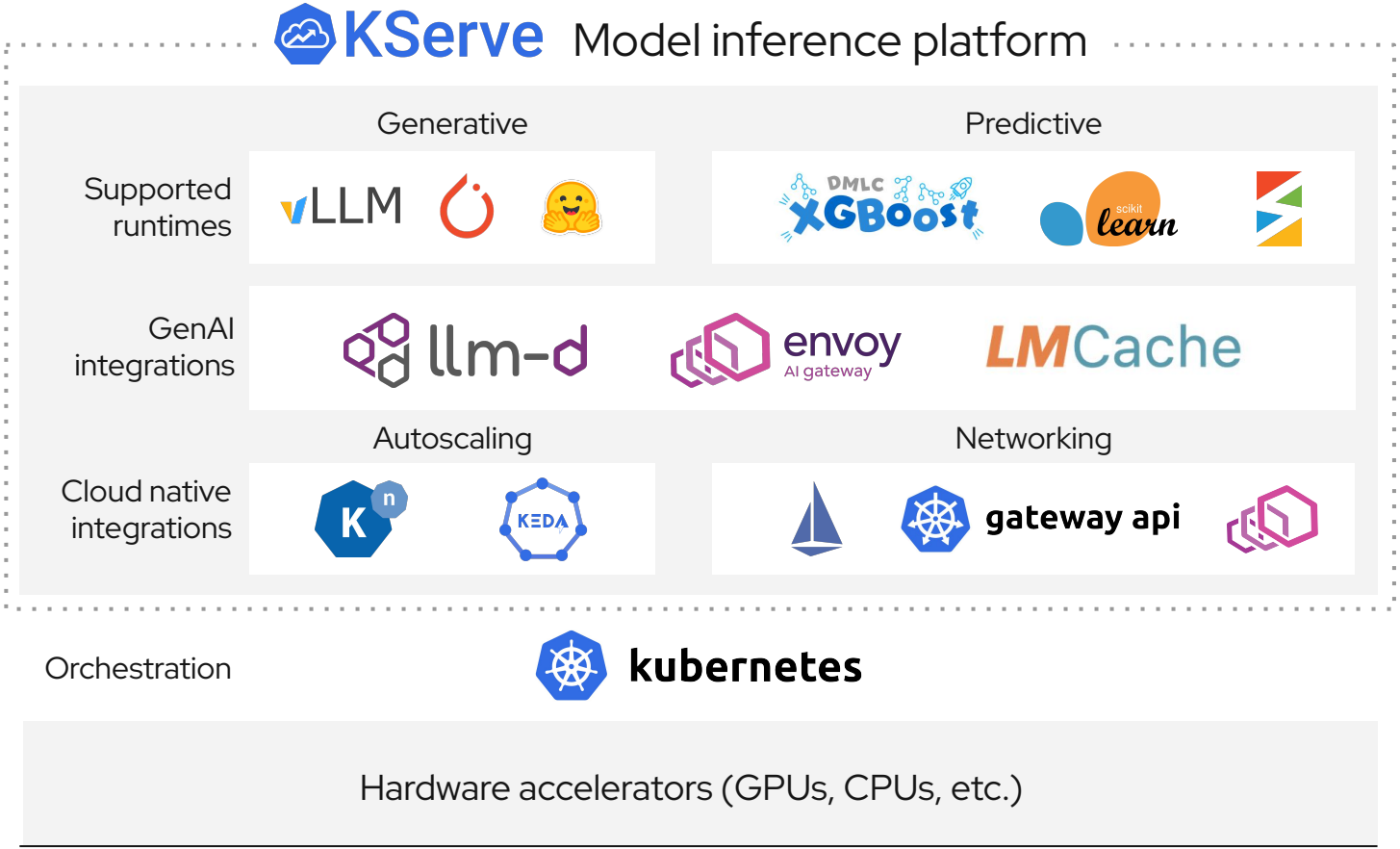




InferenceService



LLMInferenceService


KServe
Model inference platform
Generative
Predictive
Supported runtimes
vLLM
DMLC XGBoost
scikit learn
GenAI integrations
llm-d
envoy AI gateway
LMCache
Autoscaling
Networking
Cloud native integrations
KEDA
gateway api
Orchestration
kubernetes
Hardware accelerators (GPUs, CPUs, etc.)

Example

InferenceService (Classic ML)

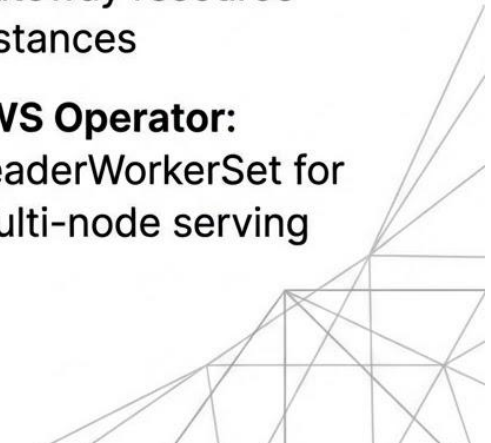
```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sklearn-iris
spec:
  predictor:
    model:
      modelFormat:
        name: sklearn
      storageUri: gs://kfserving-
examples/models/sklearn/1.0/model
```

LLMInferenceService (GenAI)

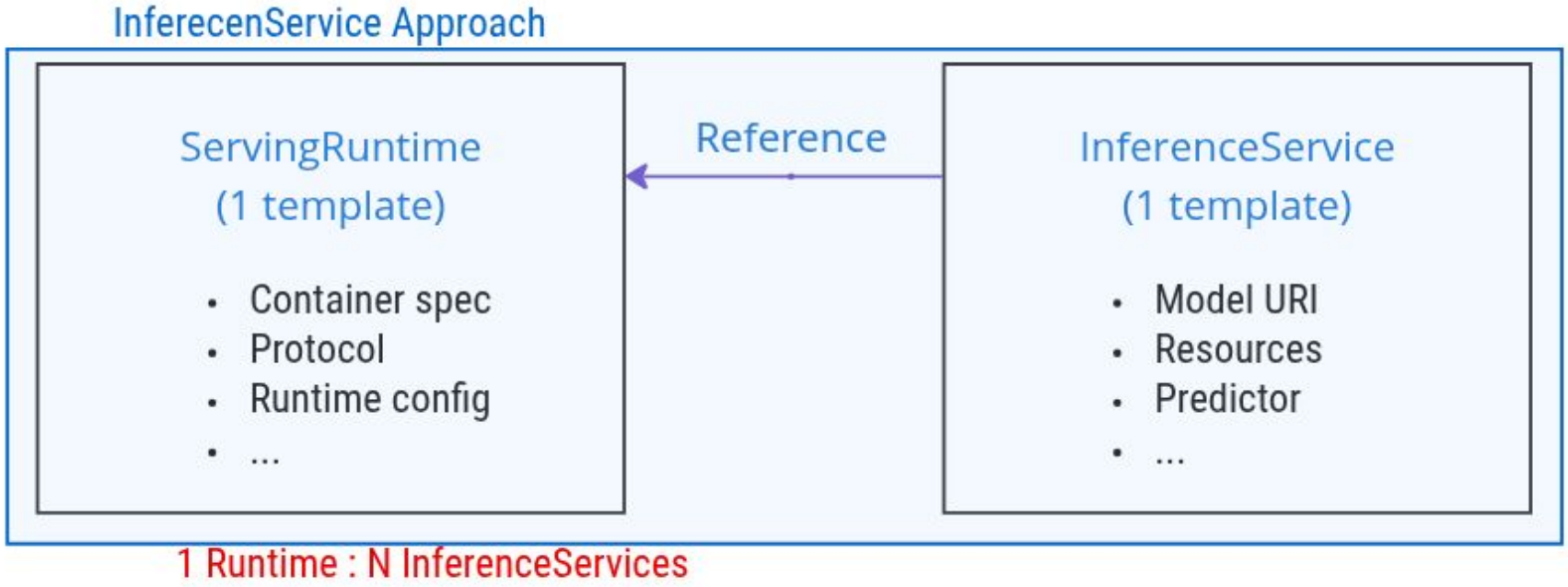
```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
  name: facebook-opt-125m-single
spec:
  model:
    uri: hf://facebook/opt-125m
    name: facebook/opt-125m
```

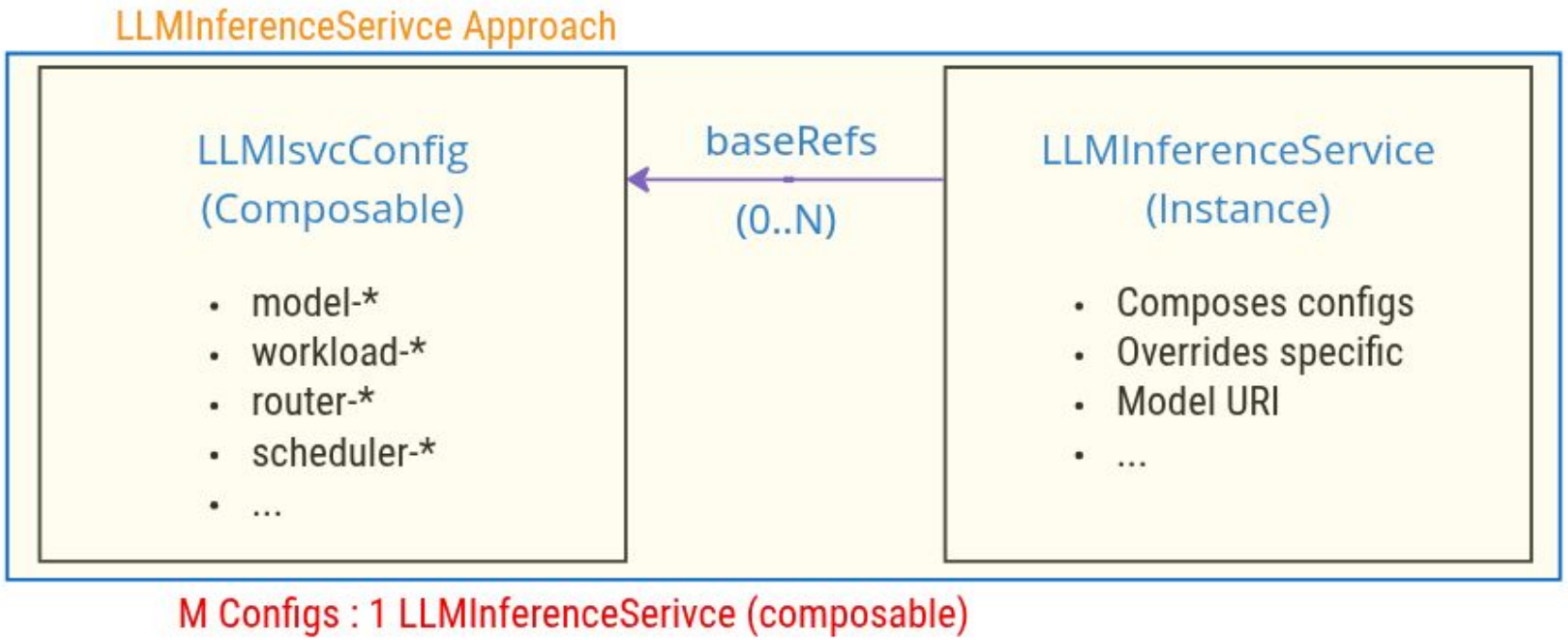
LLMISvc Dependencies

- **External LB:** Load balancer for Kind/Minikube
- **cert-manager:** TLS certificate management
- **Gateway API Extension CRDs:** Inference-specific Gateway API extensions
- **Gateway API CRDs:** Core Gateway API resources
- **Envoy Gateway:** Gateway API implementation
- **Envoy AI Gateway:** AI-specific gateway extensions
- **Gateway API GatewayClass:** Gateway class configuration
- **Gateway API Gateway:** Gateway resource instances
- **LWS Operator:** LeaderWorkerSet for multi-node serving



Reference


InferecenService Approach
ServingRuntime
(1 template)
Container spec
Protocol
Runtime config
...
Reference
InferenceService
(1 template)
Model URI
Resources
Predictor
...
1 Runtime : N InferenceServices


LLMInferenceSerivce Approach
LLMIsvcConfig
(Composable)
model-*
workload-*
router-*
scheduler-*
...
baseRefs
(0..N)
LLMInferenceService
(Instance)
Composes configs
Overrides specific
Model URI
...
M Configs : 1 LLMInferenceSerivce (composable)

LLM Inference Workload

Single-Node/Multi-GPU

SLM (less than 70B)



Single-Node Workload

spec.template
(Deployment)

- single pod per replica
- No distributed parallelism
- Suitable for small models

Multi-Node/Multi-GPU: Distributed Inference

For large models



Multi-Node Workload

spec.template
+spec.worker
(LeaderWorkerSet)

- Leader pod (head/master)
- Worker pods (N worker per replicas)
- Tensor/Pipeline/Data parallelism
- Suitable for large models

Disaggregated Prefill/Decode

High Throughput Requirement



Disaggregated Workload

spec.prefill
(Prefill pool)

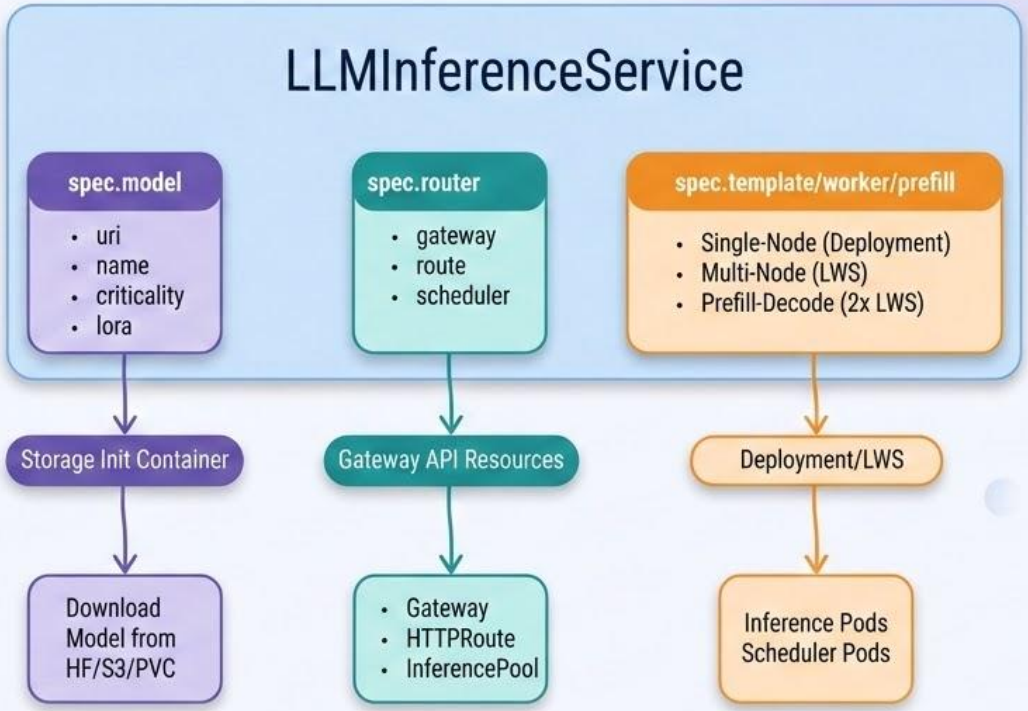
spec.template
(Decode pool)

- Leader pod (head/master)
- Process prompts
- Generate KV cache
- High concurrency

- Generate tokens
- Use KV cache
- Sequential decode

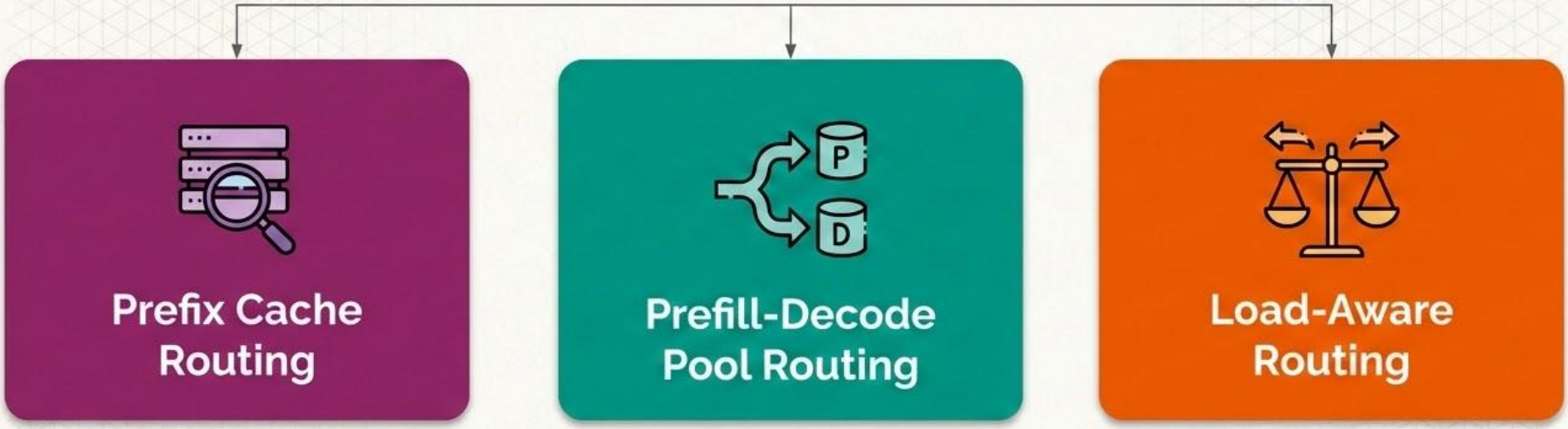
LLM Inference Control Plane API

- **Model Spec:** Model URL, LORA
- **Router Spec:** Gateway API, Scheduler
- **Workload Spec:** Single/Multi Node, Prefill/Decode
- **Parallelism Spec:** TP/DP/EP

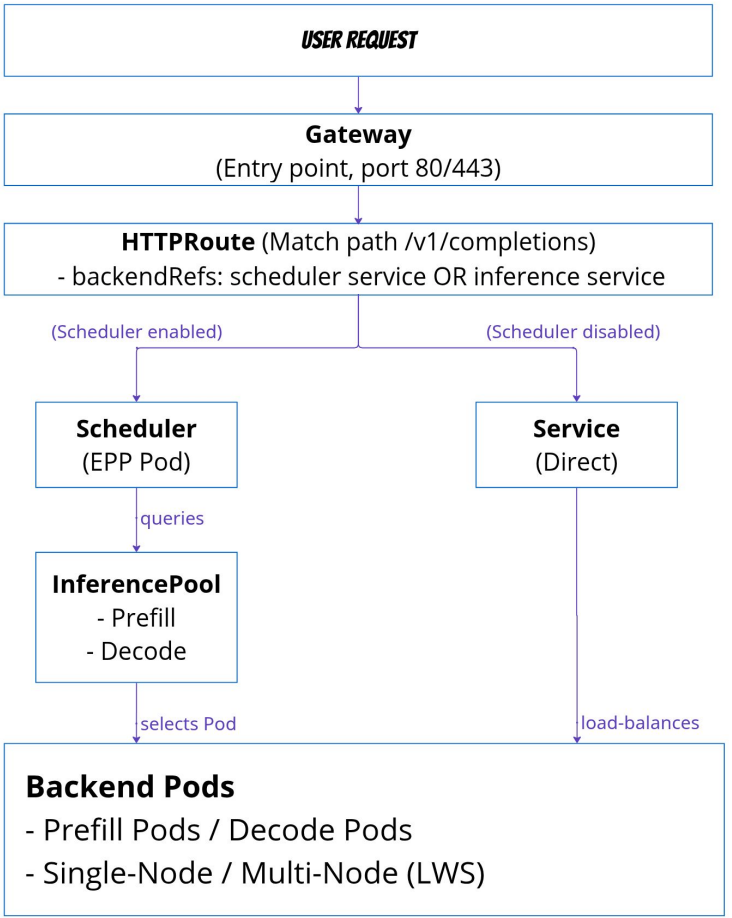


LLM Inference Data Plane

Intelligent Endpoint Picking



LLM Inference Data Plane FLOW



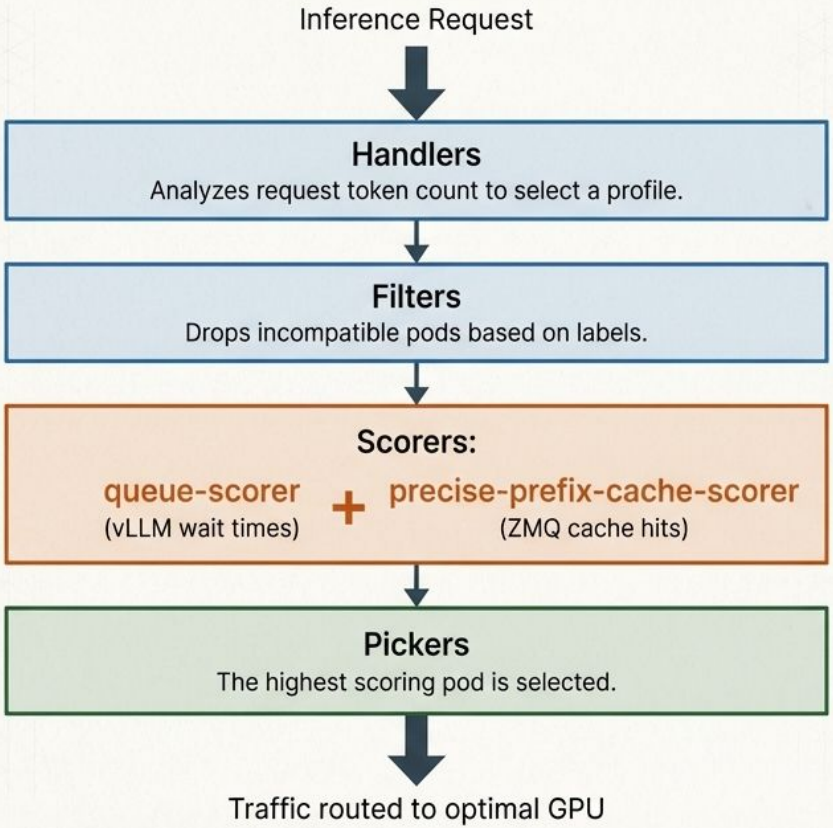
Thank you

kserve website: https://kserve.github.io/website/
kserver github: https://github.com/kserve/kserve
KServe CNCF slack: https://cloud-native.slack.com/archives/C06AH2C3K8B

User Research: Model Serving Experience with KServe



The Inference Gateway introduces model-aware routing and KV cache locality



NotebookLM